

Database Management System

ECS 403

**Lecture 29,30: Concurrency control and
Recovery Management: transaction model
properties, state serializability**

By

Ms. Bapita Roy

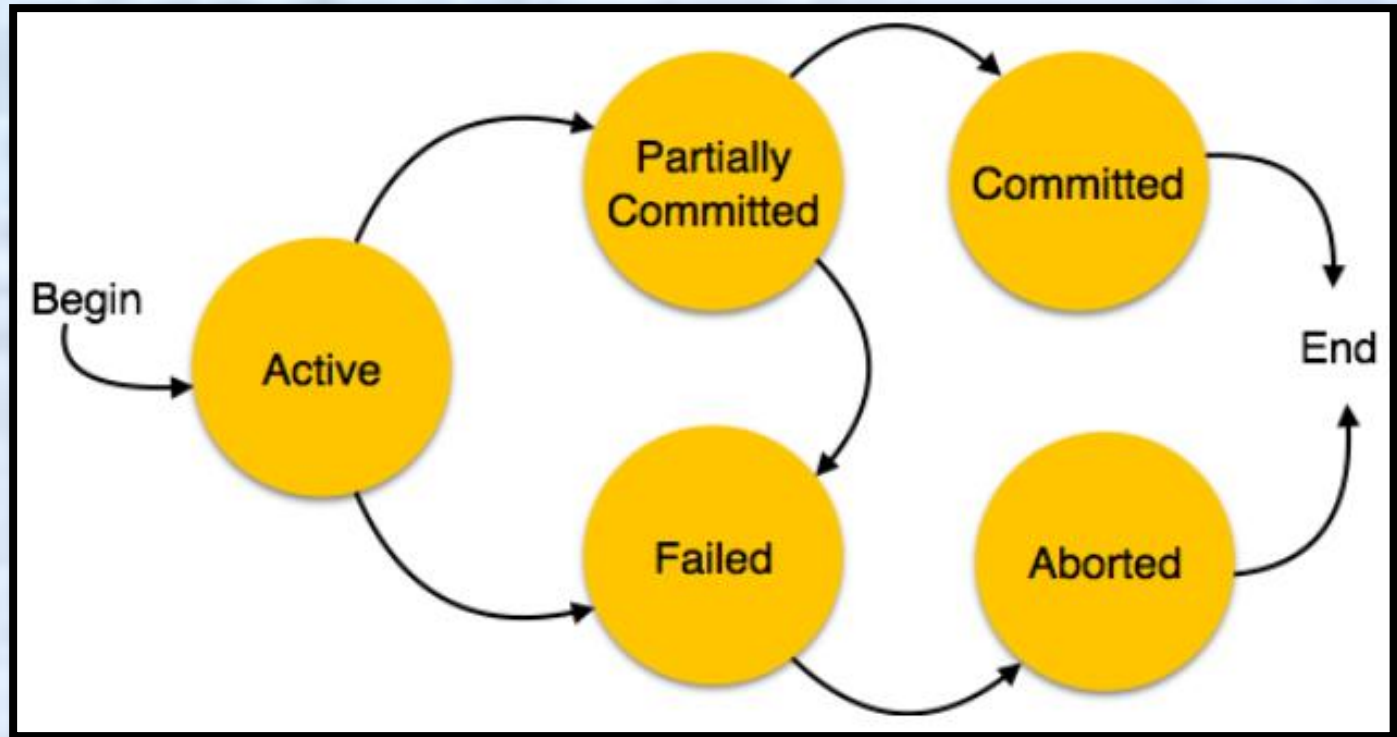
Head of the department

Electronics & Computer Science Engineering

Guru Nanak Institute of Technology

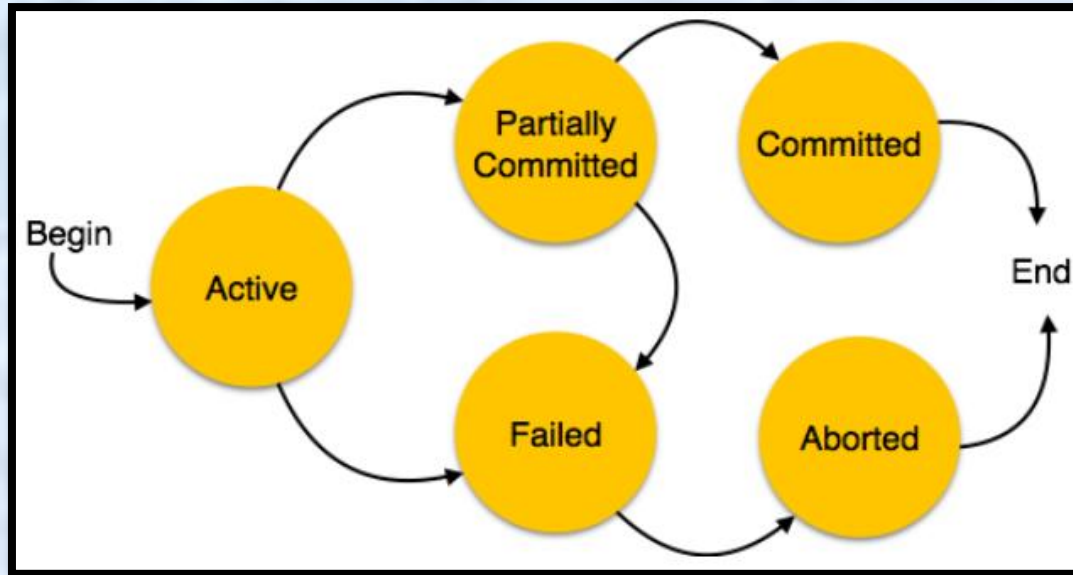
Contact no: 9231893138, Mail Id: head_ecs.gnit@gmail.com

Transaction States



- **Active** – In this state, the transaction is being executed. This is the initial state of every transaction.
- **Partially Committed** – When a transaction executes its final operation, it is said to be in a partially committed state.

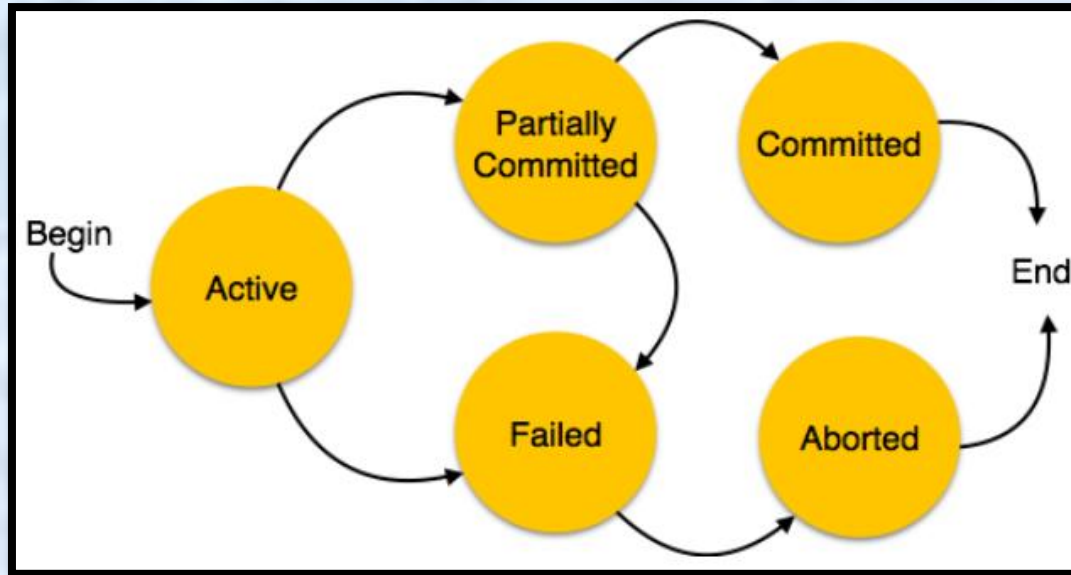
Transaction States



➤ **Failed** – A transaction is said to be in a failed state if any of the checks made by the database recovery system fails. A failed transaction can no longer proceed further.

➤ **Aborted** – If any of the checks fails and the transaction has reached a failed state, then the recovery manager rolls back all its write operations on the database to bring the database back to its original state where it was prior to the execution of the transaction. Transactions in this state are called aborted.

Transaction States



The database recovery module can select one of the two operations after a transaction aborts –

- ✓ Re-start the transaction
- ✓ Kill the transaction

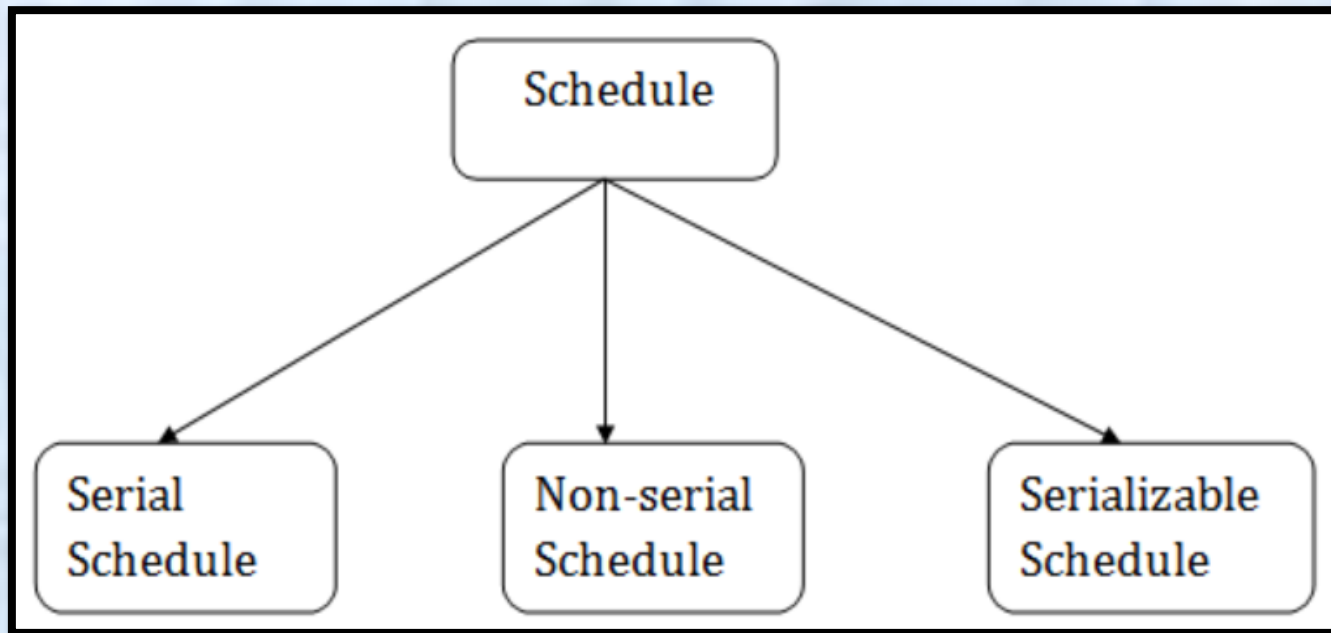
➤ **Committed** – If a transaction executes all its operations successfully, it is said to be committed. All its effects are now permanently established on the database system.

Scheduling

A series of operation from one transaction to another transaction is known as schedule.

A chronological execution sequence of a transaction is called a schedule.

A schedule can have many transactions in it, each comprising of a number of instructions/tasks.



Scheduling

Serial Schedules-

In serial schedules,

All the transactions execute serially one after the other.

When one transaction executes, no other transaction is allowed to execute.

Characteristics-

Serial schedules are always-

Consistent

Recoverable

Scheduling

Example-01:

Transaction T1	Transaction T2
R (A)	
W (A)	
R (B)	
W (B)	
Commit	
	R (A)
	W (B)
	Commit

In this schedule,

There are two transactions T1 and T2 executing serially one after the other.

Transaction T1 executes first.

After T1 completes its execution, transaction T2 executes.

So, this schedule is an example of a Serial Schedule.

Scheduling

Non-Serial Schedules-

In non-serial schedules,

Multiple transactions execute concurrently.

Operations of all the transactions are interleaved or mixed with each other.

Characteristics-

Non-serial schedules are NOT always-

Consistent

Recoverable

Scheduling

Example-01:

Transaction T1	Transaction T2
R (A)	
W (B)	
	R (A)
R (B)	
W (B)	
Commit	
	R (B)
	Commit

In this schedule,

There are two transactions T1 and T2 executing concurrently.

The operations of T1 and T2 are interleaved.

So, this schedule is an example of a Non-Serial Schedule.

Scheduling

Finding Number of Schedules-

Consider there are n number of transactions T1, T2, T3 , Tn with N1, N2, N3 , Nn number of operations respectively.

1. Total Number of Schedules-

Total number of possible schedules (serial + non-serial) is given by-

$$\frac{(N_1 + N_2 + N_3 + \dots + N_n)!}{N_1! \times N_2! \times N_3! \times \dots \times N_n!}$$

Scheduling

2.Total Number of Serial Schedules-

Total number of serial schedules

= Number of different ways of arranging n transactions

= $n!$

3.Total Number of Non-Serial Schedules-

Total number of non-serial schedules

= Total number of schedules – Total number of serial schedules

Scheduling

Example:

Consider there are three transactions with 2, 3, 4 operations respectively, then find-

How many total number of schedules are possible?

How many total number of serial schedules are possible?

How many total number of non-serial schedules are possible?

Scheduling

Solution-

Total Number of Schedules-

Using the above formula, we have-

Total Number of Serial Schedules-

$$\begin{aligned}\text{Total number of schedules} &= \frac{(2 + 3 + 4)!}{2! \times 3! \times 4!} \\ &= 1260\end{aligned}$$

Total number of serial schedules

= Number of different ways of arranging 3 transactions

= 3!

= 6

Total Number of Non-Serial Schedules-

Total number of non-serial schedules

= Total number of schedules – Total number of serial schedules

= 1260 – 6

= 1254

Serializable schedule-

The serializability of schedules is used to find non-serial schedules that allow the transaction to execute concurrently without interfering with one another.

A non-serial schedule will be Serializable if its result is equal to the result of its transactions executed serially.

There are two types of Serializable Schedules:

➤ Conflict Serializable Schedules

and

➤ View Serializable Schedules

Conflict Serializable:

A schedule is called conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.

Conflicting operations: Two operations are said to be conflicting if all conditions satisfy:

- **They belong to different transactions**
- **They operate on the same data item**
- **At Least one of them is a write operation**

Conflict Serializable:

Example: –

Conflicting operations pair ($R1(A)$, $W2(A)$) because they belong to two different transactions on same data item A and one of them is write operation.

Similarly, ($W1(A)$, $W2(A)$) and ($W1(A)$, $R2(A)$) pairs are also conflicting.

On the other hand, ($R1(A)$, $W2(B)$) pair is non-conflicting because they operate on different data item.

Similarly, ($W1(A)$, $W2(B)$) pair is non-conflicting.

To check the conflict serializability of a schedule using Precedence Graph:

Precedence Graph or Serialization Graph is used commonly to test Conflict Serializability of a schedule.

It is a directed Graph (V, E) consisting of a set of nodes $V = \{T_1, T_2, T_3, \dots, T_n\}$ and a set of directed edges $E = \{e_1, e_2, e_3, \dots, e_m\}$.

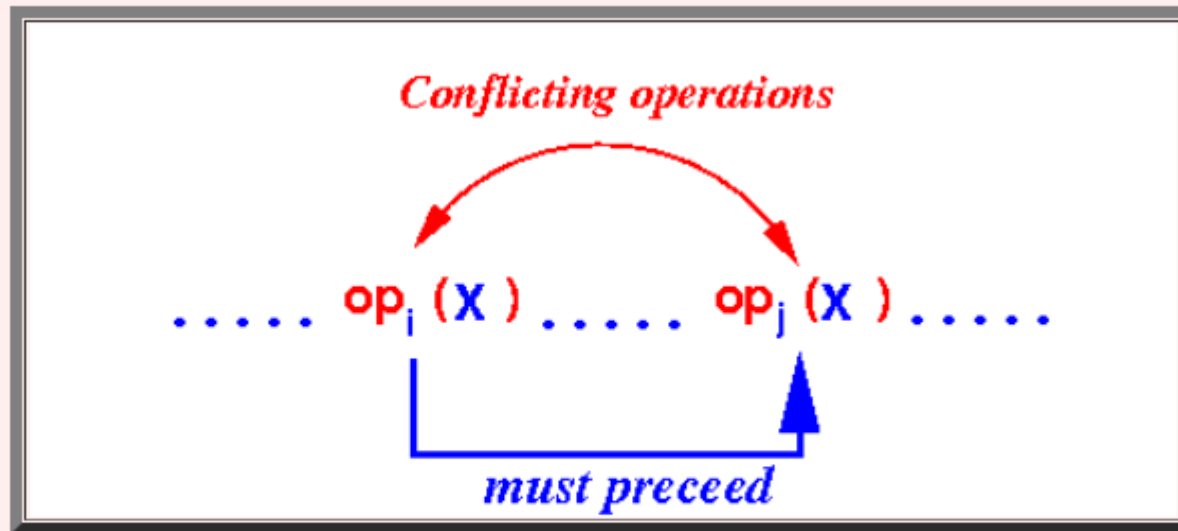
The graph contains one node for each Transaction T_i .

An edge e_i is of the form $T_j \rightarrow T_k$ where T_j is the starting node of e_i and T_k is the ending node of e_i .

An edge e_i is constructed between nodes T_j to T_k if one of the operations in T_j appears in the schedule before some conflicting operation in T_k .

To check the conflict serializability of a schedule using Precedence Graph:

- If we find 2 *conflicting actions* (anywhere in a schedule):

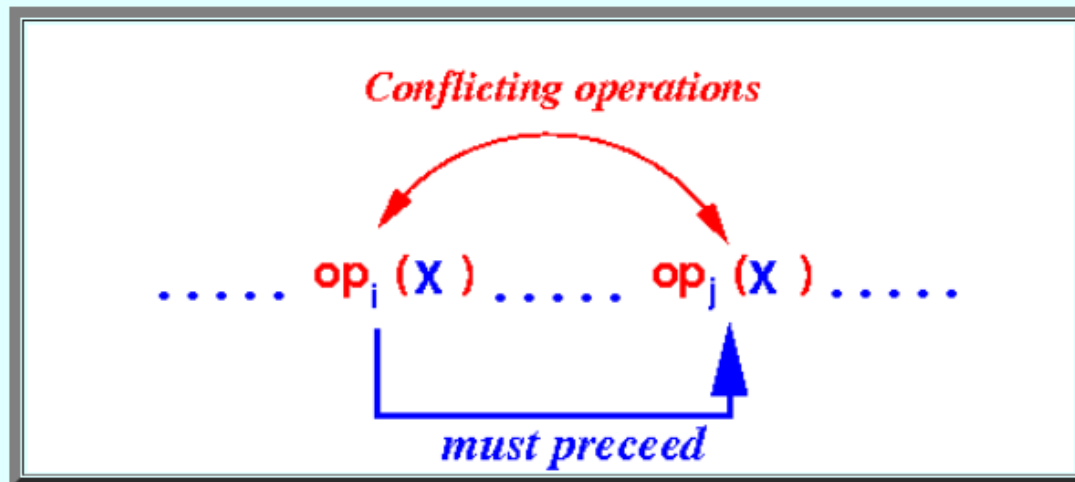


then (because $op_j(X)$ can *never* be *swap* with $op_i(X)$):

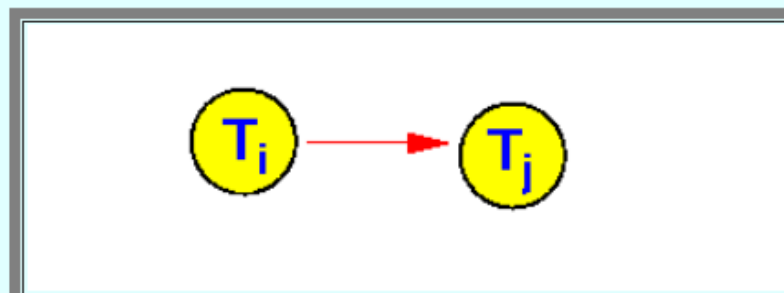
- Transaction T_i **must precede** transaction T_j in a *serial schedule*

To check the conflict serializability of a schedule using Precedence Graph:

- For *every* pair of *conflicting* actions in a schedule *S*:



we add the edge $T_i \rightarrow T_j$ to the graph



To check the conflict serializability of a schedule using Precedence Graph:

The Schedule S is serializable if there is no cycle in the precedence graph.

For example,

Consider the schedule S :

S : r1(x) r1(y) w2(x) w1(x) r2(y)

- i. r(X) conflict is w(X) : It appears in T2 after T1
- ii. So a Edge will be there from T1 to T2
- iii. r(Y) conflict is w(Y) : It appears in T2 after T1
- iv. So a Edge will be there from T1 to T2.

It is already there, so no need to draw again

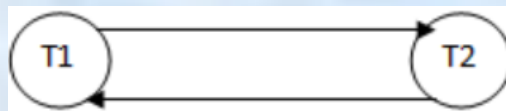
- v. w(X) conflict is w(X) and r(X) : w(X) appears in T1 after T2, but there is no r(X) in T1 after T2

- vi. So a Edge will be there from T2 to T1

- vii. w(X) conflict is w(X) and r(X) : but there is no w(X) or r(X) in T1 after T2

- viii. r(Y) conflicts w(Y): but there is no step after this step in the schedule

T1	T2
r(X)	
r(Y)	
	w(X)
w(X)	
	r(Y)

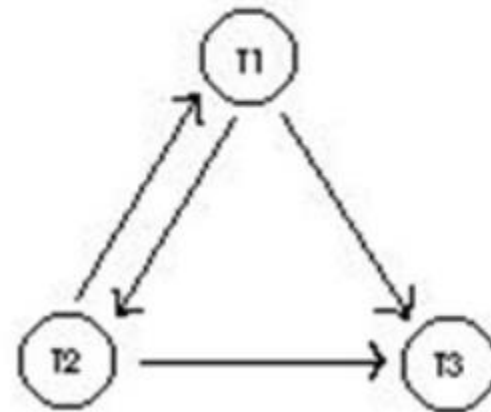


Since the graph is cyclic, we can conclude that it is **not conflict serializable** to any schedule serial schedule.

Precedence Graph Example 1

- Compute a precedence graph for schedule B (right)
- three vertices (T1, T2, T3)
- edge from T_i to T_j if
 - T_i writes Q before T_j reads Q
 - T_i reads Q before T_j writes Q
 - T_i writes Q before T_j writes Q

Schedule B		
T1	T2	T3
read(Q)		
	write(Q)	
write(Q)		
		write(Q)



Example:

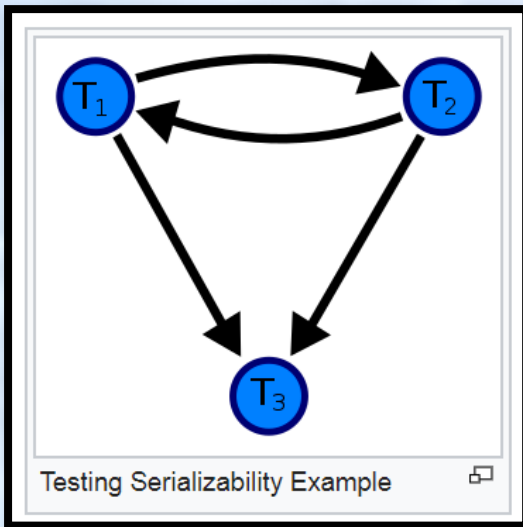
For each transaction T_x participating in schedule S , create a node labeled T_i in the precedence graph. Thus the precedence graph contains T_1, T_2, T_3 .

For each case in S where T_j executes a *read_item(X)* after T_i executes a *write_item(X)*, create an edge ($T_i \rightarrow T_j$) in the precedence graph. This occurs nowhere in the above example, as there is no read after write.

For each case in S where T_j executes a *write_item(X)* after T_i executes a *read_item(X)*, create an edge ($T_i \rightarrow T_j$) in the precedence graph. This results in a directed edge from T_1 to T_2 (as T_1 has *R(A)* before T_2 having *W(A)*).

For each case in S where T_j executes a *write_item(X)* after T_i executes a *write_item(X)*, create an edge ($T_i \rightarrow T_j$) in the precedence graph. This results in directed edges from T_2 to T_1 , T_2 to T_3 and T_1 to T_3 .

The schedule S is serializable if and only if the precedence graph has no cycles. As T_1 and T_2 constitute a cycle, the above example is **not (conflict) serializable**.

$$S = \begin{bmatrix} T1 & T2 & T3 \\ R(A) & & \\ & W(A) & \\ & Com. & \\ W(A) & & \\ Com. & & \\ & & W(A) \\ & & Com. \end{bmatrix}$$


THANK YOU